Auditing the Blast Radius: Artifact Consistency in npm Supply-Chain Prioritization

Anonymous Authors

***Abstract*—Dependency reach, shared maintainers, and public continuous integration workflows offer useful but incomplete views of software supply-chain exposure. Their combination into a predictive score requires internally consistent artifacts and a defensible outcome population. We audit an npm research repository containing 25,228 packages and two compressed metric artifacts. Independent recomputation reproduces all seven database metric fields, yet the companion CSV disagrees on workflow reach for 17,809 packages: only 20 have nonzero values in the CSV, compared with 17,809 in the database. At a one-percent review budget of 253 packages, their equal-weight score queues share only 65 entries. Large score ties further affect queue membership. Separately, the incident companion contains 47 package-event records and 45 retained pre-event source objects, but supplies no cutoff-valid control population. These findings establish artifact inconsistency and evaluation ineligibility, rather than predictive accuracy. We provide an offline audit, aggregate results, and an evidence-aware protocol separating structural exposure, workflow configuration, release authority, and future documented compromise. The contribution is a reproducible measurement study and benchmark specification; no learned model or attacker-return estimate is claimed.**

***Index Terms*—software supply chain, npm, dependency graphs, artifact consistency, defensive prioritization, temporal validity**

I. Introduction

Dependency graphs distribute both functionality and trust. Prior npm research establishes that dependency concentration, maintainer sharing, and metadata weaknesses create systemic review concerns [1], [2]. Turning these observations into a defensive ranking requires consistent inputs and an outcome population appropriate to the claim being tested.

We study a research repository whose collector combines npm metadata, dependency edges, public GitHub workflow-directory observations, and advisories. Its implemented score weights maintainer portfolio size and the number of downstream packages associated with public workflows. Project documents propose graph learning, but the available implementation does not train or evaluate that model.

The repository contains compressed CSV and SQLite metric artifacts with the same package population. We find that their plain dependency and maintainer metrics agree, while workflow-derived values differ substantially. A shared repository revision or population count therefore does not guarantee a coherent snapshot. An analysis can reproduce the CSV faithfully yet misdescribe the database stored beside it.

We investigate three questions: **RQ1**, whether stored metrics reproduce and agree across artifacts; **RQ2**, how artifact choice, weighting, and boundary ties change a fixed-budget review queue; and **RQ3**, which prediction claims the incident evidence supports.

The contributions are an independently executed offline metric audit, a quantitative analysis of review-queue drift, and an evidence-based assessment of benchmark readiness. For 25,228 packages, all seven database metric fields reproduce, but workflow reach differs across artifacts for 17,809 packages. Their equal-weight top-253 queues share only 65 entries. The incident companion supplies event records without eligible controls, so the paper reports measurement results and a proposed evaluation protocol rather than predictive accuracy.

II. Related Work and Novelty Boundary

Structure and detection. Zimmermann et al. study npm dependency and maintainer concentration; Zahan et al. examine metadata weak links [1], [2]. These establish the feature families rather than their novelty here. Ohm et al. organize documented supply-chain attacks [3]. Amalfi and a recent detector benchmark evaluate released package content [4], [5]. A benign release-classification label does not establish that its publisher remains uncompromised in a future horizon.

Workflows and authority. CI workflow characterization and ARGUS analyze security properties and injection paths beyond directory presence [6], [7]. GitHub documents permission and privileged-trigger semantics [8]. Release-authority measurement compares predecessor release paths and reports no exact malicious-version overlap with its policy triggers in the studied cohort [9]. Authority discontinuity is consequently a distinct review surface.

Historical data and forecasting. npm-follower preserves releases that can later disappear, and CHRONO-RESOLUTION provides release-point dependency resolution [10], [11]. SupplyTGN evaluates temporal vulnerability-risk ranking in a PyPI/OSV setting; PatchRisk studies future transitive vulnerability exposure for npm and PyPI [12], [13]. These narrow any broad claim to introduce temporal package forecasting.

Table I locates our empirical contribution: disagreement between co-located artifacts and its queue consequences. We claim neither a new graph architecture nor the first temporal package benchmark. The literature screen, updated on 20 October 2026, is targeted rather than systematic.

III. Graph Metrics and Evidence

A dependency edge $(u, v) \in E$ points from dependent package u to dependency v. The implementation projects stored version pairs onto package names. For observed packages P, its reverse transitive dependent set is

$$\mathcal{D}(p) = \{q \in P \setminus \{p\} : q \rightsquigarrow p\}. \tag{1}$$

TABLE I
POSITIONING AGAINST THE CLOSEST RESEARCH STREAMS. DIFFERENCES IDENTIFY SCOPE, NOT SUPERIORITY.

Work	Principal evidence	Evaluated target or artifact	Boundary for this study
Small World; Weak Links [1], [2]	Dependencies, maintainers, metadata	Concentration and metadata weaknesses	Structural descriptors already established
Amalfi; detector benchmark [4], [5]	Released package content	Malicious-package detection	Detection labels do not define future non-events
CI characterization; ARGUS [6], [7]	Workflow and action code	Configuration properties; injection paths	Directory presence is a weaker observation
On Good Authority [9]	Predecessor release-path evidence	Authority discontinuities	Authority change and compromise differ
npm-follower; CHRONO-RESOLUTION [10], [11]	Archived releases; release-point resolution	Historical data resources	Graph reconstruction alone is not new
SupplyTGN; PatchRisk [12], [13]	Temporal package/advisory evidence	Future vulnerability-risk ranking	Temporal forecasting is already studied
This study	Two local archives; incident-record inventory	Metric consistency, queue drift, feasibility	No fitted model or predictive benchmark result

Let $M(p)$ be the observed maintainer set and $d(m)$ its member's package portfolio size. The implemented statistics are

$$W_{\max}(p) = \max_{m \in M(p)} d(m), \qquad W_{\Sigma}(p) = \sum_{m \in M(p)} d(m), \quad (2)$$

with zero for an empty maintainer set. Summed portfolios can overlap; registry identities do not establish historical publication authority.

The workflow flag $h(q)$ is one if any stored version of package q has a positive workflow-directory observation. Workflow reach is

$$B(p) = \sum_{q \in \mathcal{D}(p)} h(q). \quad (3)$$

It counts packages associated with directory observations, not distinct organizations, actual installations, or privileged executions. Multiple packages may map to one repository.

The field named `attacker_roi` stores the legacy score

$$S_{\alpha}(p) = \alpha \frac{W_{\max}(p)}{\max_q W_{\max}(q)} + (1-\alpha)\frac{B(p)}{\max_q B(q)}. \quad (4)$$

Zero denominators become one. The default is $\alpha = 0.5$. Neither component observes success probability or attack cost, so no attacker-return interpretation is established.

Structural exposure, workflow configuration, release-authority change, and future documented compromise are different decision targets. For a future event time T_p, cutoff t, and horizon H, the proposed outcome is

$$Y_{p,t,H} = \mathbb{1}\{T_p \in (t, t+H]\}. \quad (5)$$

Unknown outcomes remain unknown. A reviewed zero means no documented qualifying event under a stated surveillance protocol, not intrinsic benignness.

Each feature needs source, capture time, validity interval, fidelity, and missingness. A later registry query is not automatically an immutable historical snapshot. Likewise, a byte checksum establishes source identity rather than historical availability. We separate measured audit findings, repository-record inventories, and proposed experiments throughout.

IV. AUDIT METHODOLOGY

A. Inputs and Independent Reconstruction

The study uses a compressed metric CSV and SQLite archive supplied together. The database contains 25,228 packages, 22,340 package-version rows, 12,217 maintainers, 79,405 package–maintainer relations, 121,885 dependency rows, and 153,488 advisories. High-impact seeding and dependency discovery produce a convenience sample, not an npm census. The audit date, 20 October 2026, is distinct from collection time.

The offline audit extracts SQLite temporarily and opens it read-only. It verifies package uniqueness and equal populations, derives maintainer portfolios, projects dependency edges onto package-name pairs, and traverses reverse edges with duplicate suppression. The starting package is excluded even in cycles. Workflow flags and all seven metrics are reconstructed independently of the production program. Integer values are compared exactly and database scores at absolute tolerance 10^{-12}.

CSV scores have six decimal places. Cross-artifact comparisons therefore use tolerance 5.00001×10^{-7} and separately compare database scores rounded to six decimals. Input hashes, code, and aggregate results are retained in the audit record. No package is installed, payload retrieved, or source archive modified.

B. Queues, Ties, and Sensitivity

For budget $K = \lceil 0.01|P| \rceil = 253$, let $Q_K^A(f)$ denote the top-K set under field f in artifact A. Scores descend; ties use ascending Unicode code-point package-name order. We report intersection and Jaccard overlap,

$$J(Q, Q') = \frac{|Q \cap Q'|}{|Q \cup Q'|}, \quad (6)$$

and repeat with descending names. These comparisons measure consistency rather than security effectiveness.

If a packages exceed the boundary score, b tie there, and $s = K - a$ slots remain, uniform tie breaking includes a tied

TABLE II
CROSS-ARTIFACT COMPARISON. POSITIVE COUNTS AND PACKAGE-LEVEL DISAGREEMENTS; SCORE DIFFERENCES ALLOW CSV ROUNDING.

Field	CSV > 0	DB > 0	Different
Maintainer portfolio (max)	22,147	22,147	0
Maintainer portfolio (sum)	22,147	22,147	0
Workflow reach	20	17,809	17,809
Direct dependents	20,331	20,331	0
Workflow direct dependents	15	15,017	15,014
Transitive dependents	20,328	20,328	0
Legacy score	22,147	24,502	17,809

package with probability s/b. Inclusion probabilities above and below the boundary are one and zero. Independently randomizing each queue's ties gives

$$\mathbb{E}[|Q \cap Q'|] = \sum_{p \in P} \pi_p \pi'_p. \tag{7}$$

This is combinatorial sensitivity, not a confidence interval.

We sweep $\alpha \in \{0, 0.1, \ldots, 1\}$ using each artifact's own maxima without outcome tuning. Spearman correlations use average ranks. Separately, we inventory incident-companion CSVs, count records and family identifiers, and compare recorded source timestamps with cutoffs. We do not repeat incident adjudication or authenticate the original registry responses.

V. EMPIRICAL RESULTS

A. RQ1: Metric and Artifact Consistency

All seven database metric fields reproduce with zero mismatches under the declared tolerances. The two archives contain identical sets of 25,228 package names. Their maintainer portfolio, direct-dependent, and transitive-dependent values agree for every package. The workflow-derived quantities do not (Table II).

The CSV has 20 nonzero workflow-reach entries (0.0793% of packages), while the database has 17,809 (70.5922%). Their maxima are two and 3,310, respectively. Workflow-bearing direct-dependent counts disagree for 15,014 packages; 15 CSV entries and 15,017 database entries are positive. The equal-weight score differs for the same 17,809 packages as workflow reach, even after accounting for six-decimal CSV serialization. This pattern localizes the disagreement to workflow-derived metrics in these artifacts; it does not establish when or why the divergence occurred.

The database contains 15,300 positive workflow observations across version rows, 6,223 checked negatives, and 817 unchecked rows. Positive observations cover 15,233 distinct packages. Therefore the sparse CSV workflow reach cannot be interpreted as evidence that workflow information is generally absent from the stored database. Figure 1 contrasts the artifact-level summaries.

The platform snapshot manifest assigns a June 2026 snapshot date, while every stored metric row carries `2026-07-14 06:39:51` as its calculation timestamp. These describe different events, and the repository does not supply enough capture history to establish a coherent historical collection time for all inputs. We use archive digests as identities and report this timing limitation explicitly. We neither overwrite the supplied exports nor silently select a shared historical date.

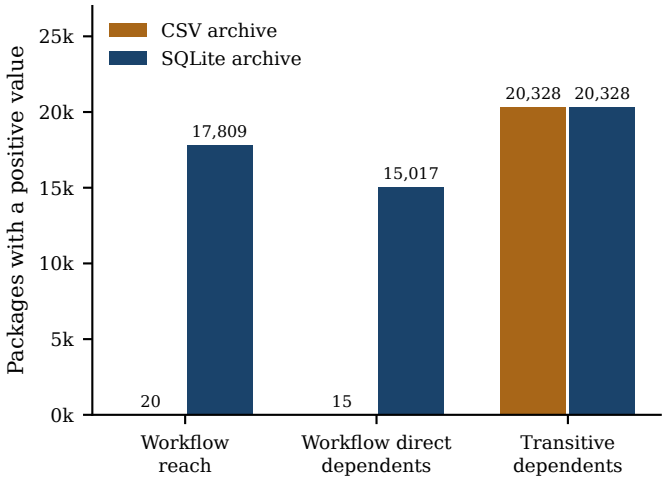


Fig. 1. Positive metric counts from the two archives. The same package population and unchanged dependency metrics coexist with large workflow-derived disagreement.

TABLE III
CROSS-ARTIFACT TOP-253 QUEUE OVERLAP UNDER THE SPECIFIED NAME ORDERING. EXPECTED OVERLAP INDEPENDENTLY RANDOMIZES BOUNDARY TIES.

Queue	Shared	Jaccard	Expected shared
Equal-weight score	65	0.1474	55.14
Dependency reach	253	1.0000	253.00
Workflow reach	20	0.0412	22.15

B. RQ2: Queue Drift at a Fixed Review Budget

At $K = 253$, the equal-weight score queues intersect in 65 packages, or 25.69% of each queue. Their Jaccard overlap is 0.1474. Thus 188 selected entries change when choosing one archive rather than the other. The workflow-reach queues share 20 packages and have Jaccard overlap 0.0412. By contrast, the dependency-reach queues match exactly (Table III).

Boundary ties qualify these exact set comparisons. In the CSV score queue, only 16 packages exceed the boundary value of 0.5; 967 tie there and compete for 237 remaining slots. The database queue has 250 above-boundary packages and six tied packages competing for three slots. Reversing the name ordering changes the shared score count from 65 to 53. Equation 7 gives an expected count of 55.14 under independent uniform tie breaking. These are alternative queue constructions, not repeated statistical samples.

For CSV workflow reach, all 20 positive entries fit within the budget and the remaining 233 slots come from 25,208 zeros. Hence most selected entries have no positive signal from that field. A deterministic top-K list can conceal this lack of resolution. Review interfaces should report score tiers and the

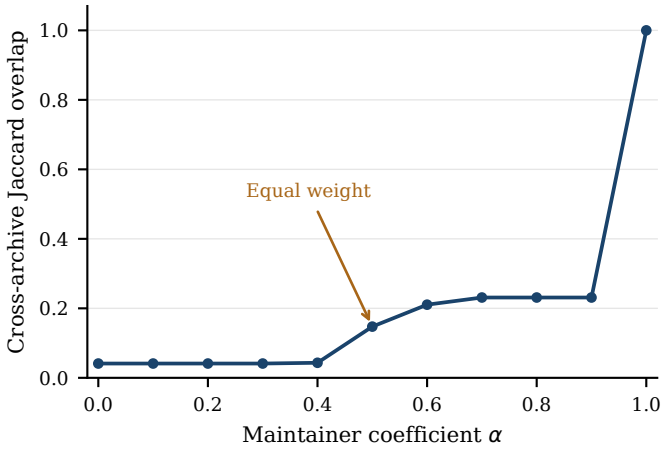


Fig. 2. Cross-artifact queue overlap as the maintainer coefficient increases. Each artifact uses its own normalization maxima. The matching endpoint removes the inconsistent workflow component.

number of tied slots rather than implying a meaningful strict order.

The weight sweep demonstrates that the disagreement persists beyond one coefficient (Figure 2). For $\alpha = 0$ through 0.3, shared membership is 20/253; at 0.5 it is 65/253; and at 0.9 it is 95/253. At $\alpha = 1$, the queues match because the score depends only on the identical maintainer component. This is a consistency endpoint, not evidence that a maintainer-only queue is operationally superior. We do not optimize weights against incident labels.

Within each archive, the legacy score also differs from dependency reach. Their top-253 intersections are 16 for the CSV and 63 for the database. Spearman correlations are 0.0825 and 0.2521, respectively. These describe distinct rankings and the influence of artifact choice. They do not show that either score predicts compromise, or that agreement with centrality is a desirable outcome.

C. Concentration and Graph-Semantics Checks

The top 123 maintainers, representing the rounded-up one-percent maintainer budget, account for 18,027 of 79,405 package–maintainer relations (22.70%). The largest portfolio contains 967 packages. These measured concentrations motivate attention to shared identities, while leaving actual publication authority and compromise likelihood unresolved.

The 121,885 stored dependency rows reduce to 88,113 distinct package-name pairs. The collector stores the destination node's `DIRECT`/`INDIRECT` status from a deps.dev root query in each row's `relation` field. Official API semantics define that status relative to the queried root, whereas graph edges identify the node declaring a dependency [14]. Consequently, filtering stored rows to `relation=DIRECT` would not generally recover all direct edges of the union graph. Our audit follows the implemented projection over every stored edge; it makes no claim of historical installed-tree fidelity. More broadly, projecting multiple versions can join paths that no single installation realized.

D. RQ3: Temporal Evidence Supports an Inventory, Not a Fit

The current snapshot companion contains 47 package-event records across 31 family identifiers, of which 45 have source objects tagged `registry_reconstructed_exact`. Those retained records span 30 family identifiers and contribute 258 runtime dependency declarations. Their recorded source publication timestamps precede their cutoffs. This check establishes consistency among the recorded fields; it does not independently authenticate their source responses or reconstruct historical workflow and authority states.

The repository supplies no cutoff-valid non-event comparison population for those events. Its two non-retained source records remain label-only exceptions. Family identifiers are not asserted to be independent attacks: the existing research notes identify campaign relatedness, and protestware, malicious new publications, and publisher compromise also differ in target meaning. Pooling these categories without an explicit estimand would obscure what a model learned.

The larger advisory table does not remove this limitation. Only 164 advisory package names overlap the metric population, 153,422 rows carry the broad classification `compromise`, and its schema has no affected-version relation. These imported records can support candidate discovery, but their volume does not establish version-specific outcomes or a defensible non-event population.

We therefore fit no model and report no PR-AUC, Recall@K, calibration curve, or predictive confidence interval. A positive-only convenience sample cannot determine a population false-positive rate or calibrated event probability. The measured contribution remains the descriptive audit, while the next section specifies the evidence needed to evaluate the original prediction question.

VI. Discussion and Evaluation Requirements

The inconsistency affects an operational decision: choosing an archive changes 188 entries in the default 253-package review queue. Reproducibility therefore requires binding derived metrics to their source observations, not merely recording population size. Queue interfaces should expose boundary ties instead of implying a strict evidence-based order.

A future compromise benchmark should freeze its at-risk population, observation unit, cutoff, and horizon before examining outcomes. Established-package publisher compromise should be separated from malicious new publications, ordinary vulnerabilities, and protestware. Controls need comparable follow-up and documented surveillance; unresolved outcomes must remain unknown. Case–control prevalence alone cannot support population calibration.

Features require historically supported dependency resolution, maintainer/publisher semantics, repository mappings, workflow configurations, authority predecessors, and missingness. Historical resources can help where coverage matches [11]; a declared version range is not an exact installed tree [15]. A checksum cannot replace missing source-time evidence.

Compare random, metadata, reach, maintainer-portfolio, and regularized tabular baselines before graph learning. Add workflow and authority evidence separately and include missingness-only ablations. Use chronological evaluation, campaign grouping, package-disjoint sensitivity checks, and validation-only tuning. Report budget-specific precision/recall and PR-AUC against the observed base rate; calibration requires representative prevalence or justified sampling correction. Release predictions and exclusions, including null results. Existing temporal studies supply useful precedents [12], [13], but the incremental value of these features for documented publisher compromise remains unevaluated here.

VII. Threats to Validity and Data Handling

Matching recomputation validates consistency with stored inputs and the formula, not collection completeness. Artifact differences could reflect collection or export timing, or another undocumented process; the audit does not establish a cause. Normalization, review budget, rounding, and tie policy affect queues. Finite-artifact totals do not justify ecosystem-wide inference or sampling confidence intervals.

Package projection removes version constraints and can join paths that no installation realized. Workflow directories do not establish actual dependency execution or privilege. Registry identities need not correspond one-to-one with publishers or organizations. High-impact sampling, shared repositories, campaign relatedness, and non-random missingness limit generalization.

The incident companion is inventoried rather than independently re-adjudicated. Recorded pre-cutoff publication times do not authenticate the original responses or guarantee immutability. Source deletion may depend on maliciousness, creating survivor bias; archive resources [10] are not assumed to cover these events automatically. The targeted literature screen cannot establish exhaustive novelty.

The audit operates offline on supplied archives, accesses SQLite read-only, and reports aggregates. It does not acquire suspicious payloads, collect secrets, or label maintainers as malicious. Original artifacts are preserved; full provenance and the extended future protocol remain in author-side working materials. No model is fitted, and the proposed benchmark has not been evaluated.

VIII. Conclusion

Independent reconstruction reproduces seven database metric fields, yet its companion CSV disagrees on workflow reach and score for 17,809 packages. The default top-253 queues share 65 entries under a stated tie rule. Matching package populations therefore do not imply matching review decisions. The incident companion lacks eligible controls, leaving prediction and calibration unevaluated. Artifact consistency and historical outcome eligibility should precede model comparison; our audit quantifies the consequences of failing that prerequisite.

Acknowledgment

A large language model assisted literature discovery, audit-code development, manuscript drafting, and editing. Numerical results were recomputed from the supplied artifacts. Human authors must review and approve the sources, code, interpretations, and disclosure before submission.

References

[1] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *28th USENIX Security Symposium*. USENIX Association, 2019, pp. 995–1010. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/zimmerman

[2] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. Maddila, and L. Williams, "What are weak links in the npm supply chain?" arXiv:2112.10165v2, 2022, ICSE SEIP 2022; DOI: 10.1145/3510457.3513044. [Online]. Available: https://arxiv.org/abs/2112.10165v2

[3] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," arXiv:2005.09535, 2020, DIMVA 2020 preprint. [Online]. Available: https://arxiv.org/abs/2005.09535

[4] A. Sejfia and M. Schäfer, "Practical automated detection of malicious npm packages," arXiv:2202.13953, 2022, ICSE 2022; DOI: 10.1145/3510003.3510104. [Online]. Available: https://arxiv.org/abs/2202.13953

[5] W. Guo, Z. Chen, Z. Xu *et al.*, "How effective are NPM malicious package detectors? a large-scale empirical study," arXiv:2603.27549v2, 2026, revised August 5, 2026. [Online]. Available: https://arxiv.org/abs/2603.27549v2

[6] I. Koishybayev, A. Nahapetyan, R. Zachariah, S. Muralee, B. Reaves, A. Kapravelos, and A. Machiry, "Characterizing the security of GitHub CI workflows," in *31st USENIX Security Symposium*. USENIX Association, 2022, pp. 2747–2763. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/koishybayev

[7] S. Muralee, I. Koishybayev, A. Nahapetyan, G. Tystahl, B. Reaves, A. Bianchi, W. Enck, A. Kapravelos, and A. Machiry, "ARGUS: A framework for staged static taint analysis of GitHub workflows and actions," in *32nd USENIX Security Symposium*. USENIX Association, 2023, pp. 6983–7000. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/muralee

[8] GitHub, "Secure use reference," GitHub Actions documentation, accessed October 20, 2026. [Online]. Available: https://docs.github.com/en/actions/reference/security/secure-use

[9] I. Santos-Grueiro, "On good authority: Release-authority measurement for registry-mediated package ecosystems," arXiv:2606.22593v2, 2026, preprint. [Online]. Available: https://arxiv.org/abs/2606.22593v2

[10] D. Pinckney, F. Cassano, A. Guha, and J. Bell, "npm-follower: A complete dataset tracking the NPM ecosystem," arXiv:2308.12545, 2023. [Online]. Available: https://arxiv.org/abs/2308.12545

[11] I. Rahman, J. Marley, R. Paramitha, and L. Williams, "CHRONO-RESOLUTION: A dependency resolution dataset at release points for npm, PyPI, and crates.io packages," arXiv:2607.15315, 2026, ICSME 2026 Tool Demonstration and Data Showcase Track. [Online]. Available: https://arxiv.org/abs/2607.15315

[12] T. P. Tran Truong, S. Ha, and X.-B. Le, "SupplyTGN: A reproducible leakage-safe benchmark for temporal software supply-chain risk," SSRN preprint 7194447, 2026, posted July 28, 2026. [Online]. Available: https://ssrn.com/abstract=7194447

[13] A. A. Shafin and K. M. Ahmed, "PatchRisk: Forecasting future vulnerability exposure in open-source dependency networks," arXiv:2609.13719, 2026, WI-IAT 2026 preprint. [Online]. Available: https://arxiv.org/abs/2609.13719

[14] Open Source Insights, "deps.dev API v3: GetDependencies," Official API documentation, accessed October 20, 2026. [Online]. Available: https://docs.deps.dev/api/v3/

[15] npm, "package-lock.json," npm CLI v11 documentation, accessed October 20, 2026. [Online]. Available: https://docs.npmjs.com/cli/v11/configuring-npm/package-lock-json/